Haetae

An independent RPKI Relying Party in Go

The Haetae is a one-horned guardian of Korean legend that can tell truth from lies.

This one checks whether a route is telling the truth.



2026-11 · v0.1 · In trial operation · Open source (ISC)

Why another RP?

RPKI outcomes rest on a single RP

- Routers accept or drop routes based on the VRPs from the RP. An RP bug or outage is a routing outage
- With two or more independent RP implementations, one exposes the other's problems
- Yet few open-source RPs are fit for production today

Haetae is an independent implementation that shares no code with existing RPs

Routinator	Rust · NLnet Labs	Active
rpki-client	C · OpenBSD	Active
FORT	C · NIC.mx	Active
rpki-prover	Haskell · solo dev	Active
OctoRPKI	Go · Cloudflare	Archived
RIPE NCC Validator	Java · RIPE NCC	Retired

Haetae at a glance

0

Diff vs rpki-client

VRPs · ASPA · router keys (same input data)

42 s

Full validation

rpki-client: 16 min on the same cache

0

External libraries

Go standard library only

1

Single binary

Fetch · validate · RTR · API in one

Validates **all standard RPKI objects** and serves routers via **RTR v0 / v1 / v2** and systems via **HTTP API**

About 9,100 lines of Go · 37 tests (incl. race detector) · runs as a daemon on a 10-min cycle · RTR to a test router

Comparison with other RPs

As of 2026-11 · from each project's public documentation

	Haetae	Routinator	rpki-client	FORT
Developer · language	**Go**	NLnet Labs · Rust	OpenBSD · C	NIC.mx · C
License	**ISC**	BSD-3	ISC	MIT
Dependencies	**None (Go stdlib only)**	Rust crates	OpenSSL etc.	OpenSSL etc.
RTR server	**Built-in (v0 / v1 / v2)**	Built-in	Separate (StayRTR etc.)	Built-in
ASPA	**Yes**	Yes (optional)	Yes	Experimental (1.7.0)
BGPsec router keys	**Yes**	Yes (optional)	Yes	Yes
Validity · delta API	**Yes (incl. ASPA paths)**	Yes	No	No
TA constraints	**Not used (per RFC)**	Not used	Rejects violations	Not used
Deployment	**Trial operation**	Most widely used	Widely used	In production

Architecture: fetch, validate, serve

1 Fetch

- RRDP first, rsync on failure
- 32 repositories in parallel
- 5-min limit per repository

2 Validate

- TAL down through CAs to manifests
- Signatures · hashes · CRLs · validity
- Child resources within parent

3 Build

- VRPs · ASPA · router keys
- SLURM local exceptions
- Serials and deltas

4 Serve

- RTR v0 / v1 / v2
- HTTP API
- JSON · CSV files

Repeats every 10 min · keeps serving the last good result if validation fails · after a restart, RTR resumes from the last result within 5 s

RFC 6487 / 3779 / 6488 / 9286 / 9582 / 8182 / 8210 / 8416 / 8630

Validates every standard object in the repositories

No standard object type is skipped, and all ~570,000 validated objects stay queryable

TA · CA certificates
RFC 6487 · 3779

CRL
RFC 6487

Manifest
RFC 9286

ROA → VRP
RFC 9582

ASPA
draft aspa-profile

BGPsec router keys
RFC 8209

GBR contacts
RFC 6493

SPL
draft rpki-prefixlist

TAK
RFC 9691

Non-standard file types (private · test objects) are only counted

Results from three RPs

Daily comparison, 2026-11-01 · VRPs fetched and validated separately by each RP, matched one by one

Haetae

1,030,627

Reference

Routinator

1,030,627

Only in Haetae: 12
Only in Routinator: 12

rpki-client

1,030,613

Only in Haetae: 27
Only in rpki-client: 13

The remaining differences are fetch timing: the three RPs fetch minutes to tens of minutes apart, so ROAs changed in between show up as differences (0.004% of the total or less)

Same input, zero difference: with the same cache and validation time, VRPs · ASPA · router keys all match rpki-client (both on a fixed snapshot and on a cache fetched by Haetae)

Before its constraints files were updated (until 2026-10-31), rpki-client had about 2,800 fewer VRPs (slide 9)

Fast validation, short cycle

Offline validation time, same server and cache (~570,000 objects) · each run on its own



Measured 2026-10 · rpki-client 9.0 = Ubuntu 24.04 package · Haetae spreads validation across cores

1–2 min
Fetch + validate per run (10-min cycle)

5 s
RTR back after a restart

~5 GB
Daemon memory (incl. runs)

Why Haetae does not use TA constraints

rpki-client filters objects with per-RIR resource files (.constraints)

- A feature of rpki-client only, not in any RFC. The rpki-client developers create the files and ship them with it
- When a file lags behind IANA allocations, valid objects for newly allocated resources are rejected
- Measured: the file (2024-04) in the Ubuntu 24.04 package (rpki-client 9.0) lacks 2410::/12, allocated to APNIC in 2024-11

Haetae does not use constraints

As the RFCs define (same as Routinator · FORT) · RIR TAs holding all resources is standard practice

Packaged files (2024-04)

2,773 VRPs · 59 ASPAs

Valid objects rejected

After switching to latest files (2026-07)

Zero difference

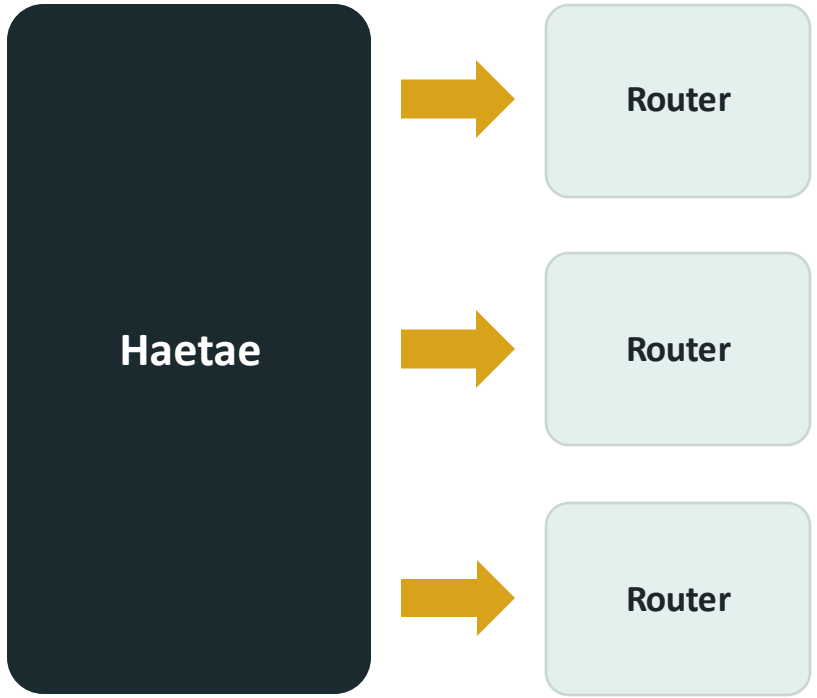
Same result as Haetae

Measured 2026-10 · same data, same rpki-client 9.0, only the constraints files changed · difference = vs. validation without constraints

RTR for routers

Serves VRPs to any router with RTR: Cisco · Juniper · Nokia · Huawei · FRR etc. (tested: Cisco IOS XR)

v0	**RFC 6810**	Even the oldest devices
v1	**RFC 8210**	Adds Router Keys (BGPsec)
v2	**draft 8210bis**	Adds ASPA (route leak prevention)
Δ	**Deltas only**	Serial Notify; serial kept over restarts
Sec	**Transport security**	IP allow list + TCP-MD5 · TCP-AO · TLS · SSH



TCP 3323 (de facto standard port)

HTTP API for systems

Follows existing RP formats where possible, and adds what they lack

Routinator-compatible

- /api/v1/validity (route validity)
- /json-delta (changes only)
- /jsonext (source ROAs, optional)

rpki-client-compatible

- /json (VRPs · ASPA · router keys)
- /csv
- Same format: comparison tools read it as is

Haetae-specific

- AS · prefix lookup, route validity
- ASPA path check (AS_PATH leaks)
- Objects · CAs, unsafe VRPs
- Expiry, run history, Prometheus

Network management systems can build status, change and expiry views and alerts from the query API and deltas (json-delta)

Built to fail safely

The input comes from untrusted repositories on the Internet

1 Untrusted input

- Limits on size · count · depth · time
- https only, no cross-host redirects
- No path traversal; a panic fails only that CA

2 When fetching fails

- Keeps using earlier data while still valid
- Falls back to rsync if RRDP keeps failing
- Deltas fully checked, then applied at once

3 When results look wrong

- If any TA fails, keeps the last result
- Warns on a large VRP drop (optional hold)
- Flags VRPs overlapping rejected CAs as unsafe

Dedicated non-root account · systemd sandboxing · RTR from allowed IPs only, optional transport security (TCP-AO etc.)

Easy to operate

One binary and one config file; settings change without a restart

1 One config file

- Every option is a config key
- Command line wins
- Unknown key or bad value: no start

2 Live reload

- Re-read on systemctl reload
- All new values checked, then applied
- On error, current values stay

3 Built-in TALs

- Official TALs of all 5 RIRs in the binary
- No dependency on other packages
- Source and key hash in startup log

4 Access control

- Allow lists for RTR and HTTP
- HTTP on multiple addresses
- Others: RTR closed · HTTP 403

systemd service · dedicated account · RTR session and serial survive restarts

Next steps

Done

New features

- RTR security (TCP-MD5 / TCP-AO / TLS / SSH)
- ASPA path check · RSC
- Config file · reload · built-in TALs

Release

v0.1 (2026-11)

- ISC license, GitHub
- English manual · API guide

Now

Trial operation

- Daemon on a 10-min cycle, results compared daily
- RTR to a test router

Next

v0.2

- Fixes from code review
- Weeks of parallel run with an existing RP
- RTR security · ASPA support per router

Haetae

Independent code · Same results · Safe failure

An RPKI Relying Party built on the Go standard library alone · ISC-licensed open source

https://github.com/yunsog/haetae